Лабораторная работа №7 "Реализация реального времени с WebSocket"

Лабораторная работа №7. Реализация реального времени с WebSocket

Ветка репозитория: `27`

---

1. ЦЕЛЬ РАБОТЫ

Освоение технологии WebSocket для создания двусторонней связи в реальном времени. Изучение библиотеки Socket.IO, создание системы обмена сообщениями или уведомлений **в соответствии с потребностями вашего курсового проекта** (чат поддержки, лайвы, уведомления о событиях, совместное редактирование и т.п.).

---

2. НЕОБХОДИМЫЕ ЗНАНИЯ

Для успешного выполнения работы студент должен знать:

- Ограничения протокола HTTP для real-time приложений.
- Принцип работы WebSocket (handshake, постоянное соединение).
- Библиотеку Socket.IO (серверная и клиентская части).
- События: `connection`, `disconnect`, `message`, `broadcast`, `join`, `leave`.

---

3. ЗАДАЧИ РАБОТЫ

- Создать сервер на Node.js + Express + Socket.IO.
- Создать клиент (React-приложение из предыдущих работ или отдельный HTML/JS) для подключения к серверу.
- Реализовать отправку и получение сообщений/уведомлений в реальном времени.

- Добавить уведомления о подключении/отключении пользователей.
- Реализовать **дополнительные функции**, соответствующие логике вашего курсового проекта (например, комнаты для разных чатов, сохранение истории в MongoDB, приватные сообщения, система голосования и т.п.).

---

4. ПОРЯДОК ВЫПОЛНЕНИЯ (общий для всех вариантов)

Часть 1. Серверная часть

1. Установите зависимости: `npm install socket.io`.
2. Создайте HTTP-сервер с Express и прикрепите Socket.IO:

   javascript

```
const http = require('http');
const express = require('express');
const { Server } = require('socket.io');

const app = express();
const server = http.createServer(app);
const io = new Server(server, { cors: { origin: '*' } });

io.on('connection', (socket) => {
  console.log('User connected', socket.id);
  // обработка ваших событий
  socket.on('chat message', (msg) => {
    io.emit('chat message', msg);
  });
  socket.on('disconnect', () => console.log('User disconnected'));
});
server.listen(3000);
```

Часть 2. Клиентская часть (React)

1. Установите клиентскую библиотеку: `npm install socket.io-client`.
2. В React-компоненте создайте соединение:

```
import io from 'socket.io-client';
const socket = io('http://localhost:3000');
```

3. Отправляйте сообщения: `socket.emit('chat message', text)`.
4. Получайте сообщения: `socket.on('chat message', (msg) => {...})`.

Часть 3. Дополнительные функции (в соответствии с вашим проектом)

Выберите **одну или несколько** (по необходимости):

- **Комнаты** – разделение пользователей по группам (например, разные заказы, разные товары).
- **Приватные сообщения** – отправка сообщения конкретному пользователю по `socket.id`.
- **Сохранение истории сообщений** в MongoDB (с использованием Mongoose).
- **Уведомления о наборе текста** (typing event).
- **Голосования / опросы** в реальном времени.
- **Синхронизация состояния** (например, совместное редактирование списка задач).

Часть 4. Тестирование

- Запустите сервер и клиентское приложение.
- Откройте несколько окон браузера (или разных браузеров) для имитации разных пользователей.
- Проверьте обмен сообщениями, уведомления о подключении/отключении, работу комнат и других реализованных функций.

---

5. СОДЕРЖАНИЕ ОТЧЕТА

Отчёт должен содержать:

- Титульный лист, цель работы.
- Схему работы WebSocket (текстовое описание или диаграмма).
- Листинг серверной части (обработчики событий).
- Листинг клиентской части (React-компонент).

- Скриншоты работы приложения с двумя окнами (имитация разных пользователей).
- Демонстрация работы дополнительных функций (комнаты, история и т.п.).
- Ответы на контрольные вопросы (16 вопросов).
- Выводы.

---

6. ТРЕБОВАНИЯ К СДАЧЕ

- Отчёт (PDF) загружается в СЭО.
- Ссылка на репозиторий GitHub с кодом (ветка `27`).
- **Индивидуальное выполнение. Вариант Темы** определяется по **Генератору вариантов.**
- Код должен быть загружен в репозиторий с веткой `27`.

---

7. КОНТРОЛЬНЫЕ ВОПРОСЫ (16 вопросов)

1. В чём отличие WebSocket от HTTP? Почему WebSocket подходит для real-time, а HTTP – нет?
2. Какую библиотеку вы использовали и почему? В чём преимущество Socket.IO перед нативным WebSocket?
3. Что такое «комнаты» (rooms) в Socket.IO? Как создать комнату и отправить сообщение только в неё?
4. Как отправить сообщение всем клиентам, кроме отправителя? Приведите пример.
5. Как обработать разрыв соединения (`disconnect`) на сервере и клиенте?
6. Как реализовать событие «печатает» (typing) без отправки самих сообщений?
7. Как в Socket.IO передать дополнительные данные (например, имя пользователя) при подключении?
8. Как сохранить историю сообщений в MongoDB с использованием Mongoose? Какие поля нужны для вашего проекта?

9. Как отправить приватное сообщение конкретному пользователю по его `socket.id`?
10. Как в Socket.IO реализовать подтверждение получения сообщения (acknowledgement)?
11. Как масштабировать Socket.IO-приложение на несколько серверов (например, с использованием Redis)? (теоретически)
12. Какие события генерируются автоматически при потере соединения и переподключении?
13. Как ограничить размер сообщения или количество сообщений в секунду (rate limiting) в Socket.IO?
14. Как защитить Socket.IO-сервер от несанкционированного подключения (например, проверка токена)?
15. Как реализовать паузу в отправке сообщений при медленном соединении (backpressure) в Socket.IO?
16. В чём разница между `socket.emit`, `io.emit` и `socket.broadcast.emit`?

---

8. ПРИМЕЧАНИЯ

- Работа выполняется **индивидуально**.
- Socket.IO автоматически использует WebSocket, но при необходимости переключается на long polling – это удобно для учебных проектов.
- Для хранения истории сообщений (если требуется) рекомендуется использовать MongoDB Atlas (бесплатный кластер M0) – он доступен в Беларуси.
- Клиент можно реализовать в виде отдельного React-компонента в вашем основном проекте (ветка `27`). Или, если не хотите смешивать, создать отдельную ветку `27-websocket` с упрощённым клиентом для демонстрации.

---

9. КРИТЕРИИ ОЦЕНКИ

- Реализация базового WebSocket-сервера и клиента (установка соединения, отправка/приём сообщений).

- Уведомления о подключении/отключении пользователей.
- Выполнение дополнительных функций, соответствующих логике курсового проекта (комнаты, история, приватные сообщения и т.д.).
- Качество кода: структура проекта, обработка ошибок, комментарии.
- Полнота отчёта: скриншоты, листинги, схема взаимодействия.
- Ссылка на репозиторий с веткой `27` и README с инструкцией по запуску.